Шаблон описания сервисов

- Описание
- Ссылки
- Требования к ресурсам
- Отказоустойчивость
- Предельные ограничения работы сервиса
- Работа сервиса
- Авторизация
- Описание окружений
    - Prod окружение
    - Stage окружение
    - Test окружение
- Сборка
- Деплой
- Конфигурация
- Переменные окружения для CI/CD
- Мониторинг
    - Мониторинг OS Linux
        - Items
        - Triggers
    - Мониторинг Docker сервиса
        - Items
        - Triggers
    - Мониторинг Application
- Логирование
- Сетевая связность
- Доступы к сервису

Описание

данный раздел *заполняет Разработчик*

Текстовое описание + должна быть таблица с описанием технологии(й) разработки

Название	**Описание**
Тип приложения	*Пример:* ASP.NET Core REST API
Прикладное ПО / Frameworks	*Пример:* .NET 7.0
Системное ПО	*Пример:* Linux
Swagger	ссылка если применимо
Архитектура	*Пример:* микросервисная или сервисная или иное

Ссылки

данный раздел *заполняет Разработчик*

	Production	**Stage**	**Test**
Сервис	Например [redacted]		
Логи	Например [redacted]		
	[redacted]		

Требования к ресурсам

данный раздел *заполняет Разработчик*

Пример: сервис запускается в контейнере, для своей работы требует 10mb оперативной памяти и 2 ядра CPU и столько-то мегабайт дискового пространства

Отказоустойчивость

данный раздел *заполняет Разработчик совместно с Руководителем проекта*

Пример:

- сервис работает в рабочее время 5/8 в одном экземпляре. Это значит, что администраторы, если "упадет" сервер, не гарантируют поддержку в нерабочее время;

- сервис работает в рабочее время 5/8, но в нескольких экземплярах (горизонтальное масштабирование);

- сервис работает непрерывно 24/7, как минимум в трех экземплярах на серверах, расположенных в разных ЦОД (центр обработки данных).

Предельные ограничения работы сервиса

данный раздел *заполняет Руководитель проекта и Разработчик*

Необходимо указать, с какими предельными значениями тестировался сервис.

Пример:

- количество запросов в секунду, который обработает сервис;
- количество информации в единицу времени, которое мы готовы принимать (например, файлы с роликами);
- количество одновременно приходящих пользователей.

Исходя из этих параметров будут установлены ограничения на продуктивных серверах или непосредственно в коде сервисов или приложений.

Например, в коде сервиса Admetrix установили ограничения на количество запросов в день.

Работа сервиса

данный раздел *заполняет Разработчик*

Пример: сервис принимает на вход запросы по rest api или по протоколу grpc или иное минимальное описание его работы в 2-х,3-х предложениях

Авторизация

данный раздел *заполняет Разработчик если авторизация поддерживается.*

Пример:

Сервис использует авторизацию по jwt-токену. То есть при обращении к сервису запрос должен содержать заголовок:

Authorization: Bearer <token> и ....

Описание окружений

данный раздел заполняют Разработчик совместно с DevOps инженерами и сетевыми администраторами

Prod окружение

Можно описать в виде таблицы, добавить текстовое описание или отобразить на схеме

Пример: в виде текстового описания

От сервиса:

- [redacted]
- [redacted]
- `< >  () (HTTP 80 )`

К сервису:

- [redacted]

Stage окружение

Можно описать в виде таблицы, добавить текстовое описание или отобразить на схеме. Если нет – написать "Отсутствует".

Test окружение

Можно описать в виде таблицы, добавить текстовое описание или отобразить на схеме. Если нет – написать "Отсутствует".

Сборка

данный раздел заполняют Разработчик совместно с DevOps инженерами

Пример:

[redacted] .gitlab-ci.yml

[redacted]

Переменная ASPNETCORE_CERT_PASS передается в докер файл и содержит пароль, необходимый для генерации само-подписанного сертификата.

Запускаемый сервис через конфигурацию получает путь до файла сертификата в системе и пароль из переменной ASPNETCORE_CERT_PASS.

Деплой

данный раздел заполняет DevOps инженер

Пример:

Сервис собирается в docker через GitLab и выкладывается на виртуальную машину

- build (сборка приложение в контейнер)
- container_scanning (проверка с помощью Trivy), если найдена уязвимость уровня HIGH или выше, то deploy не пройдет. В отчете pipe line будет указана проблема
- deploy после удачных шагов, gitlab-runner принесет собранный docker контейнер на VM и запустит его

Конфигурация

данный раздел *заполняет Разработчик*

Пример: Настройка происходит через переменные окружения и через файлы с конфигурацией. Файл конфигурации по-умолчанию - appsettings.json, остальные файлы конфигурации существуют для каждого окружения в отдельности и переопределяют настройки, заданные в файле по-умолчанию. Настройки в переменных окружения имеют приоритет над настройками в файлах конфигурации.

Переменные окружения для CI/CD

Имя переменной	**Prod**	**Stage**	**Test**	**Описание**
ПРИМЕР: Connection string				*Строка соединения с базой данных*
var2				
varN				

Мониторинг

данный раздел заполняет Разработчик совместно с Руководителем проекта

*Н*ужно описать механизм мониторинга через вызов из Prometheus (https://prometheus.io/docs/introduction/overview/)

Технический мониторинг: падение сервера и/или сервиса, отсутствие связи, потребление памяти, CPU и т.д.

Мониторинг ОС VM осуществляется через zabbix-agent2

Мониторинг OS Linux

Items

Name	Description	Type	Key and additional info
Linux: Version of Zabbix agent running		Zabbix agent	agent.version **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Host name of Zabbix agent running		Zabbix agent	agent.hostname **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Zabbix agent ping	The agent always returns 1 for this item. It could be used in combination with nodata() for availability check.	Zabbix agent	agent.ping
Linux: Zabbix agent availability	Monitoring the availability status of the agent.	Zabbix internal	zabbix[host,agent,available]
Linux: Number of CPUs		Zabbix agent	system.cpu.num **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Load average (1m avg)		Zabbix agent	system.cpu.load[all,avg1]
Linux: Load average (5m avg)		Zabbix agent	system.cpu.load[all,avg5]
Linux: Load average (15m avg)		Zabbix agent	system.cpu.load[all,avg15]
Linux: CPU utilization	The CPU utilization expressed in %.	Dependent item	system.cpu.util **Preprocessing** • JavaScript: `//Calculate utilization return (100 - value)`
Linux: CPU idle time	The time the CPU has spent doing nothing.	Zabbix agent	system.cpu.util[,idle]
Linux: CPU system time	The time the CPU has spent running the kernel and its processes.	Zabbix agent	system.cpu.util[,system]
Linux: CPU user time	The time the CPU has spent running users' processes that are not niced.	Zabbix agent	system.cpu.util[,user]
Linux: CPU nice time	The time the CPU has spent running users' processes that have been niced.	Zabbix agent	system.cpu.util[,nice]

Linux: CPU iowait time	The amount of time the CPU has been waiting for I/O to complete.	Zabbix agent	system.cpu.util[,iowait]
Linux: CPU steal time	The amount of "stolen" CPU from this virtual machine by the hypervisor for other tasks, such as running another virtual machine.	Zabbix agent	system.cpu.util[,steal]
Linux: CPU interrupt time	The amount of time the CPU has been servicing hardware interrupts.	Zabbix agent	system.cpu.util[,interrupt]
Linux: CPU softirq time	The amount of time the CPU has been servicing software interrupts.	Zabbix agent	system.cpu.util[,softirq]
Linux: CPU guest time	Guest time - the time spent on running a virtual CPU for a guest operating system.	Zabbix agent	system.cpu.util[,guest]
Linux: CPU guest nice time	The time spent on running a niced guest (a virtual CPU for guest operating systems under the control of the Linux kernel).	Zabbix agent	system.cpu.util[,guest_nice]
Linux: Context switches per second		Zabbix agent	system.cpu.switches **Preprocessing** • Change per second
Linux: Interrupts per second		Zabbix agent	system.cpu.intr **Preprocessing** • Change per second
Linux: Get filesystems	The `vfs.fs.get` key acquires raw information set about the file systems. Later to be extracted by preprocessing in dependent items.	Zabbix agent	vfs.fs.get
Linux: Memory utilization	The percentage of used memory is calculated as `100-pavailable`.	Dependent item	vm.memory.utilization **Preprocessing** • JavaScript: `return (100-value);`
Linux: Available memory in %	The available memory as percentage of the total. See also Appendixes in Zabbix Documentation about parameters of the `vm.memory.size` item.	Zabbix agent	vm.memory.size[pavailable]
Linux: Total memory	The total memory expressed in bytes.	Zabbix agent	vm.memory.size[total]
Linux: Available memory	The available memory: - in Linux - available = free + buffers + cache; - on other platforms calculation may vary. See also Appendixes in Zabbix Documentation about parameters of the `vm.memory.size` item.	Zabbix agent	vm.memory.size[available]
Linux: Total swap space	The total space of the swap volume/file expressed in bytes.	Zabbix agent	system.swap.size[,total]
Linux: Free swap space	The free space of the swap volume/file expressed in bytes.	Zabbix agent	system.swap.size[,free]
Linux: Free swap space in %	The free space of the swap volume/file expressed in %.	Zabbix agent	system.swap.size[,pfree]
Linux: System uptime	The system uptime expressed in the following format: "N days, hh:mm:ss".	Zabbix agent	system.uptime
Linux: System boot time		Zabbix agent	system.boottime **Preprocessing** • Discard unchanged with heartbeat: `1h`
Linux: System local time	The local system time of the host.	Zabbix agent	system.localtime
Linux: System name	The host name of the system.	Zabbix agent	system.hostname **Preprocessing** • Discard unchanged with heartbeat: `12h`

Linux: System description	The information as normally returned by `uname -a`.	Zabbix agent	system.uname **Preprocessing** • Discard unchanged with heartbeat: `12h`
Linux: Number of logged in users	The number of users who are currently logged in.	Zabbix agent	system.users.num
Linux: Maximum number of open file descriptors	It could be increased by using `sysctl` utility or modifying the file `/etc/sysctl.conf`.	Zabbix agent	kernel.maxfiles **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Maximum number of processes	It could be increased by using `sysctl` utility or modifying the file `/etc/sysctl.conf`.	Zabbix agent	kernel.maxproc **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Number of processes		Zabbix agent	proc.num
Linux: Number of running processes		Zabbix agent	proc.num[,,run]
Linux: Checksum of /etc /passwd		Zabbix agent	vfs.file.cksum[/etc/passwd,sha256] **Preprocessing** • Discard unchanged with heartbeat: `1h`
Linux: Operating system		Zabbix agent	system.sw.os **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Operating system architecture	The architecture of the operating system.	Zabbix agent	system.sw.arch **Preprocessing** • Discard unchanged with heartbeat: `1d`
Linux: Number of installed packages		Zabbix agent	system.sw.packages.get **Preprocessing** • JSON Path: `$.length()` • Discard unchanged with heartbeat: `12h`

Triggers

Name	Description	Expression	Severity	Dependencies and additional info
Linux: Zabbix agent is not available	For passive only agents, host availability is used with {$AGENT. TIMEOUT} as time threshold.	`max(/Linux by Zabbix agent/zabbix[host,agent,available], {$AGENT.TIMEOUT})=0`	Average	**Manual close**: Yes
Linux: Load average is too high	The load average per CPU is too high. The system may be slow to respond.	`min(/Linux by Zabbix agent/system.cpu.load[all,avg1],5m)/last (/Linux by Zabbix agent/system.cpu.num)>{$LOAD_AVG_PER_CPU. MAX.WARN} and last(/Linux by Zabbix agent/system.cpu.load [all,avg5])>0 and last(/Linux by Zabbix agent/system.cpu.load [all,avg15])>0`	Average	
Linux: High CPU utilization	The CPU utilization is too high. The system might be slow to respond.	`min(/Linux by Zabbix agent/system.cpu.util,5m)>{$CPU.UTIL. CRIT}`	Warning	**Depends on**: • Linux: Load average is too high

Linux: High memory utilization	The system is running out of free memory.	`min(/Linux by Zabbix agent/vm.memory.utilization,5m)> {$MEMORY.UTIL.MAX}`	Average	**Depends on**: • Linux: Lack of available memory
Linux: Lack of available memory		`max(/Linux by Zabbix agent/vm.memory.size[available],5m)< {$MEMORY.AVAILABLE.MIN} and last(/Linux by Zabbix agent/vm. memory.size[total])>0`	Average	
Linux: High swap space usage	If there is no swap configured, this trigger is ignored.	`max(/Linux by Zabbix agent/system.swap.size[,pfree],5m)< {$SWAP.PFREE.MIN.WARN} and last(/Linux by Zabbix agent /system.swap.size[,total])>0`	Warning	**Depends on**: • Linux: Lack of available memory • Linux: High memory utilization
Linux: {HOST. NAME} has been restarted	The host uptime is less than 10 minutes.	`last(/Linux by Zabbix agent/system.uptime)<10m`	Warning	**Manual close**: Yes
Linux: System time is out of sync	The host's system time is different from Zabbix server time.	`fuzzytime(/Linux by Zabbix agent/system.localtime,{$SYSTEM. FUZZYTIME.MAX})=0`	Warning	**Manual close**: Yes
Linux: System name has changed	The name of the system has changed. Acknowledge to close the problem manually.	`change(/Linux by Zabbix agent/system.hostname) and length (last(/Linux by Zabbix agent/system.hostname))>0`	Info	**Manual close**: Yes
Linux: Configured max number of open filedescriptors is too low		`last(/Linux by Zabbix agent/kernel.maxfiles)<{$KERNEL. MAXFILES.MIN}`	Info	
Linux: Configured max number of processes is too low		`last(/Linux by Zabbix agent/kernel.maxproc)<{$KERNEL.MAXPROC. MIN}`	Info	**Depends on**: • Linux: Getting closer to process limit
Linux: Getting closer to process limit		`last(/Linux by Zabbix agent/proc.num)/last(/Linux by Zabbix agent/kernel.maxproc)*100>80`	Warning	
Linux: /etc /passwd has been changed		`last(/Linux by Zabbix agent/vfs.file.cksum[/etc/passwd, sha256],#1)<>last(/Linux by Zabbix agent/vfs.file.cksum[/etc /passwd,sha256],#2)`	Info	**Manual close**: Yes **Depends on**: • Linux: System name has changed • Linux: Operating system description has changed
Linux: Operating system description has changed	The description of the operating system has changed. Possible reasons are that the system has been updated or replaced. Acknowledge to close the problem manually.	`change(/Linux by Zabbix agent/system.sw.os) and length(last( /Linux by Zabbix agent/system.sw.os))>0`	Info	**Manual close**: Yes **Depends on**: • Linux: System name has changed

Linux: Number of installed packages has been changed		`change(/Linux by Zabbix agent/system.sw.packages.get)<>0`	Warning	**Manual close**: Yes

Мониторинг Docker сервиса

Items

Name	Description	Type	Key and additional info
Docker: Ping		Zabbix agent	docker.ping **Preprocessing** • Discard unchanged with heartbeat: `10m`
Docker: Get info		Zabbix agent	docker.info
Docker: Get containers		Zabbix agent	docker.containers
Docker: Get images		Zabbix agent	docker.images
Docker: Get data_usage		Zabbix agent	docker.data_usage
Docker: Containers total	Total number of containers on this host.	Dependent item	docker.containers.total **Preprocessing** • JSON Path: `$.Containers`
Docker: Containers running	Total number of containers running on this host.	Dependent item	docker.containers.running **Preprocessing** • JSON Path: `$.ContainersRunning`
Docker: Containers stopped	Total number of containers stopped on this host.	Dependent item	docker.containers.stopped **Preprocessing** • JSON Path: `$.ContainersStopped`
Docker: Containers paused	Total number of containers paused on this host.	Dependent item	docker.containers.paused **Preprocessing** • JSON Path: `$.ContainersPaused`
Docker: Images total	Number of images with intermediate image layers.	Dependent item	docker.images.total **Preprocessing** • JSON Path: `$.Images`
Docker: Storage driver	Docker storage driver. https://docs.docker.com/storage/storagedriver/	Dependent item	docker.driver **Preprocessing** • JSON Path: `$.Driver` • Discard unchanged with heartbeat: `1d`
Docker: Memory limit enabled		Dependent item	docker.mem_limit.enabled **Preprocessing** • JSON Path: `$.MemoryLimit` • Boolean to decimal • Discard unchanged with heartbeat: `1d`

Docker: Swap limit enabled		Dependent item	docker.swap_limit.enabled **Preprocessing** • JSON Path: `$.SwapLimit` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Kernel memory enabled		Dependent item	docker.kernel_mem.enabled **Preprocessing** • JSON Path: `$.KernelMemory` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Kernel memory TCP enabled		Dependent item	docker.kernel_mem_tcp.enabled **Preprocessing** • JSON Path: `$.KernelMemoryTCP` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: CPU CFS Period enabled	https://docs.docker.com/config/containers/resource_constraints/#configure-the-default-cfs-scheduler	Dependent item	docker.cpu_cfs_period.enabled **Preprocessing** • JSON Path: `$.CpuCfsPeriod` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: CPU CFS Quota enabled	https://docs.docker.com/config/containers/resource_constraints/#configure-the-default-cfs-scheduler	Dependent item	docker.cpu_cfs_quota.enabled **Preprocessing** • JSON Path: `$.CpuCfsQuota` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: CPU Shares enabled	https://docs.docker.com/config/containers/resource_constraints/#configure-the-default-cfs-scheduler	Dependent item	docker.cpu_shares.enabled **Preprocessing** • JSON Path: `$.CPUShares` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: CPU Set enabled	https://docs.docker.com/config/containers/resource_constraints/#configure-the-default-cfs-scheduler	Dependent item	docker.cpu_set.enabled **Preprocessing** • JSON Path: `$.CPUSet` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Pids limit enabled		Dependent item	docker.pids_limit.enabled **Preprocessing** • JSON Path: `$.PidsLimit` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: IPv4 Forwarding enabled		Dependent item	docker.ipv4_forwarding.enabled **Preprocessing** • JSON Path: `$.IPv4Forwarding` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Debug enabled		Dependent item	docker.debug.enabled **Preprocessing** • JSON Path: `$.Debug` • Boolean to decimal • Discard unchanged with heartbeat: `1d`

Docker: Nfd	Number of used File Descriptors.	Dependent item	docker.nfd **Preprocessing** • JSON Path: `$.NFd`
Docker: OomKill disabled		Dependent item	docker.oomkill.disabled **Preprocessing** • JSON Path: `$.OomKillDisable` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Goroutines	Number of goroutines.	Dependent item	docker.goroutines **Preprocessing** • JSON Path: `$.NGoroutines`
Docker: Logging driver		Dependent item	docker.logging_driver **Preprocessing** • JSON Path: `$.LoggingDriver` • Discard unchanged with heartbeat: `1d`
Docker: Cgroup driver		Dependent item	docker.cgroup_driver **Preprocessing** • JSON Path: `$.CgroupDriver` • Discard unchanged with heartbeat: `1d`
Docker: NEvents listener		Dependent item	docker.nevents_listener **Preprocessing** • JSON Path: `$.NEventsListener`
Docker: Kernel version		Dependent item	docker.kernel_version **Preprocessing** • JSON Path: `$.KernelVersion` • Discard unchanged with heartbeat: `1d`
Docker: Operating system		Dependent item	docker.operating_system **Preprocessing** • JSON Path: `$.OperatingSystem` • Discard unchanged with heartbeat: `1d`
Docker: OS type		Dependent item	docker.os_type **Preprocessing** • JSON Path: `$.OSType` • Discard unchanged with heartbeat: `1d`
Docker: Architecture		Dependent item	docker.architecture **Preprocessing** • JSON Path: `$.Architecture` • Discard unchanged with heartbeat: `1d`
Docker: NCPU		Dependent item	docker.ncpu **Preprocessing** • JSON Path: `$.NCPU`
Docker: Memory total		Dependent item	docker.mem.total **Preprocessing** • JSON Path: `$.MemTotal`

Docker: Docker root dir		Dependent item	docker.root_dir **Preprocessing** • JSON Path: `$.DockerRootDir` • Discard unchanged with heartbeat: `1d`
Docker: Name		Dependent item	docker.name **Preprocessing** • JSON Path: `$.Name`
Docker: Server version		Dependent item	docker.server_version **Preprocessing** • JSON Path: `$.ServerVersion` • Discard unchanged with heartbeat: `1d`
Docker: Default runtime		Dependent item	docker.default_runtime **Preprocessing** • JSON Path: `$.DefaultRuntime` • Discard unchanged with heartbeat: `1d`
Docker: Live restore enabled		Dependent item	docker.live_restore.enabled **Preprocessing** • JSON Path: `$.LiveRestoreEnabled` • Boolean to decimal • Discard unchanged with heartbeat: `1d`
Docker: Layers size		Dependent item	docker.layers_size **Preprocessing** • JSON Path: `$.LayersSize`
Docker: Images size		Dependent item	docker.images_size **Preprocessing** • JSON Path: `$.Images[*].Size.sum()`
Docker: Containers size		Dependent item	docker.containers_size **Preprocessing** • JSON Path: `$.Containers[*].SizeRw.sum()`
Docker: Volumes size		Dependent item	docker.volumes_size **Preprocessing** • JSON Path: `$.Volumes[*].UsageData.Size.sum()`
Docker: Images available	Number of top-level images.	Dependent item	docker.images.top_level **Preprocessing** • JSON Path: `$.length()`

Triggers

Name	Description	Expression	Severity	Dependencies and additional info
Docker: Service is down		`last(/Docker by Zabbix agent 2/docker.ping)=0`	Average	**Manual close**: Yes

Docker: Failed to fetch info data	Zabbix has not received data for items for the last 30 minutes.	`nodata(/Docker by Zabbix agent 2/docker.name,30m)=1`	Warning	**Manual close**: Yes **Depends on**: • Docker: Service is down
Docker: Version has changed	Docker version has changed. Acknowledge to close the problem manually.	`last(/Docker by Zabbix agent 2/docker.server_version,#1)<>last(/Docker by Zabbix agent 2/docker.server_version,#2) and length(last(/Docker by Zabbix agent 2/docker.server_version))>0`	Info	**Manual close**: Yes

Мониторинг Application

Приложение должно иметь endpoint в формате prometheus /metrics

Библиотеки можно посмотреть тут https://prometheus.io/docs/instrumenting/clientlibs/

Бизнес-мониторинг: реализуется с помощью метода /health, должен возвращать три состояния:

- Healthy - система "здорова"
- Degraded - снижение качества работы системы. Необходимо указать параметры качества и кому отправлять в случае несоответствия.
- Unhealthy - система не работает. Требуется перезапуск. Должен быть EndPoint, который производит перезапуск.

Руководитель проекта и разработчик должны определить бизнес-критерии и, возможно, технические критерии (например, отсутствие связи с СУБД, брокером сообщений и т.д.), которые необходимо контролировать.

Пример: падение уровня mac-адресов ниже какого-то предела, или падение RTB запросов ниже определенного значения, или какая-то операция не выполняется n-раз подряд.

см. описание примера технологии мониторинга по ссылке Мониторинг бизнес-метрик сервисов

Руководитель проекта должен указать, нужно ли ему для Технического и Бизнес-мониторинга:

- отражение в Grafana (или в др. средствах отражения информации из Prometheus в читаемом виде);
- настройка получения уведомлений заинтересованным лицам (указать куда и кому: на почту, маттермост, телеграмм, иное)

Логирование

данный раздел заполняет Руководитель проекта совместно с Разработчиком

Требуется логирование STDOUT и STDERR докер-контейнера в Elasticsearch (команда docker logs без опции --details).

С возможностью доступа через Kibana.

Сетевая связность

данный раздел заполняет Сетевой администратор

Доступы к сервису

данный раздел заполняют Сетевой администратор совместно с Разработчиком и Руководителем проекта

Пример: с точки зрения Руководителя проекта доступ к сервису должны иметь все компьютеры, на которых установлена АПР-Система, с точки зрения Сетевого администратора – это набор подсетей